

Angular (v20+) Hands-On Lab Report

Incremental Project: Student Course Portal

Hands-On:	Hands-On 1 [Beginner]
Topic:	Environment Setup, Project Structure & First Component
Developer:	Cathy George
OS Version:	Windows 11
Date:	July 22, 2026
Track:	Java FSE Angular Track

1. Objective

The primary objective of this hands-on exercise is to establish a modern Angular development environment using the Angular CLI, scaffold the Student Course Portal project structure, explore the role and purpose of crucial configuration files generated by the CLI, and build the foundation pages including the header navigation bar and the initial welcome dashboard.

2. Angular Concepts Covered

- **Angular CLI (Command Line Interface):** Used for scaffolding, building, generating code components, and running local dev servers.
- **Project Structure & Bootstrapping:** Understanding how main.ts bootstraps the root application config and component in modern standalone mode.
- **Standalone Components:** Creating components using the newer standalone pattern (default in v17/18/19/20/21) which bypasses the legacy NgModule structure.
- **Single Page Application (SPA) Routing:** Configuring path-component mappings in routes to load pages dynamically inside the router-outlet.
- **Component Styling & Encapsulation:** Organizing modular styles (scoped to header, home, etc.) alongside global style overrides in styles.css.

3. Software & Tools Used

Software / Tool	Version / URL
Node.js + npm	v22.20.0 (LTS 20+) / npm 10.9.3
Angular CLI	v21.2.16
Code Editor	VS Code with Angular Language Service Extension
Web Browser	Google Chrome + DevTools
PDF Generator	Python 3.14 + ReportLab library

4. Commands Executed

Step / Action	Command Line
Scaffold New Project	ng new student-course-portal --routing --style=css --ssr=false
Navigate to directory	cd student-course-portal
Generate Header component	ng generate component components/header
Generate Home component	ng generate component pages/home
Generate Course List component	ng generate component pages/course-list

Generate Student Profile component	ng generate component pages/student-profile
Run Application Locally	ng serve
Build Production Bundle	ng build

5. Project Folder Structure

```

student-course-portal/
  ■■■ angular.json (CLI settings)
  ■■■ package.json (dependencies)
  ■■■ tsconfig.json (TS config)
  ■■■ notes.txt (exercise documentation)
  ■■■ src/
  ■■■ index.html (main layout entry)
  ■■■ main.ts (bootstrap entry)
  ■■■ styles.css (global style variables)
  ■■■ app/
  ■■■ app.config.ts (providers & routing imports)
  ■■■ app.routes.ts (route mappings)
  ■■■ app.ts (root component logic)
  ■■■ app.html (header + router-outlet wrapper)
  ■■■ components/
  ■ ■■■ header/ (Header navbar component)
  ■■■ pages/
  ■■■ home/ (Dashboard component)
  ■■■ course-list/ (Course offering details placeholder)
  ■■■ student-profile/ (Student info placeholder)

```

6. Code Implementation Details

Root Routing Configuration (src/app/app.routes.ts)

```
import { Routes } from '@angular/router';
import { Home } from '../pages/home/home';
import { CourseList } from '../pages/course-list/course-list';
import { StudentProfile } from '../pages/student-profile/student-profile';

export const routes: Routes = [
  { path: '', component: Home },
  { path: 'courses', component: CourseList },
  { path: 'profile', component: StudentProfile },
  { path: '**', redirectTo: '' }
];
```

Root Component Class (src/app/app.ts)

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { Header } from '../components/header/header';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, Header],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  protected readonly title = signal('student-course-portal');
}
```

Header Navigation Class (src/app/components/header/header.ts)

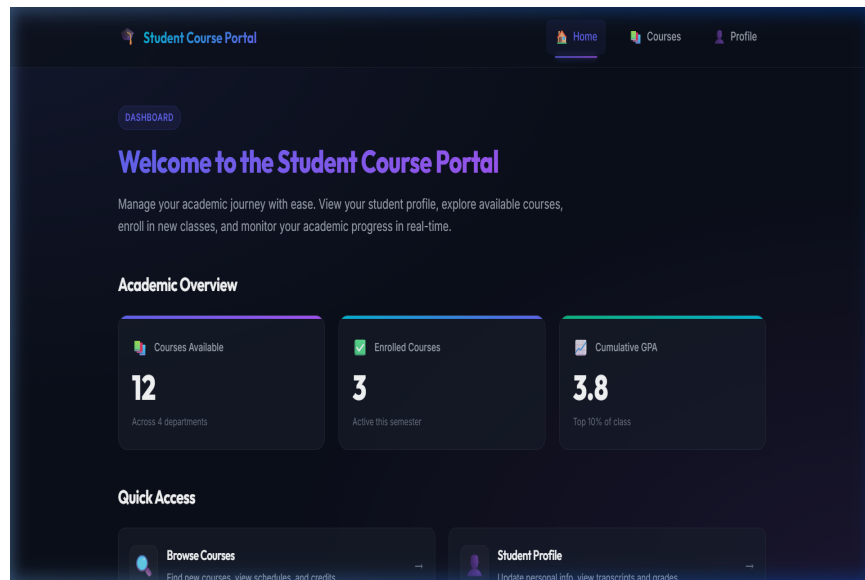
```
import { Component } from '@angular/core';
import { RouterLink, RouterLinkActive } from '@angular/router';

@Component({
  selector: 'app-header',
  imports: [RouterLink, RouterLinkActive],
  templateUrl: './header.html',
  styleUrls: ['./header.css'],
})
export class Header {}
```

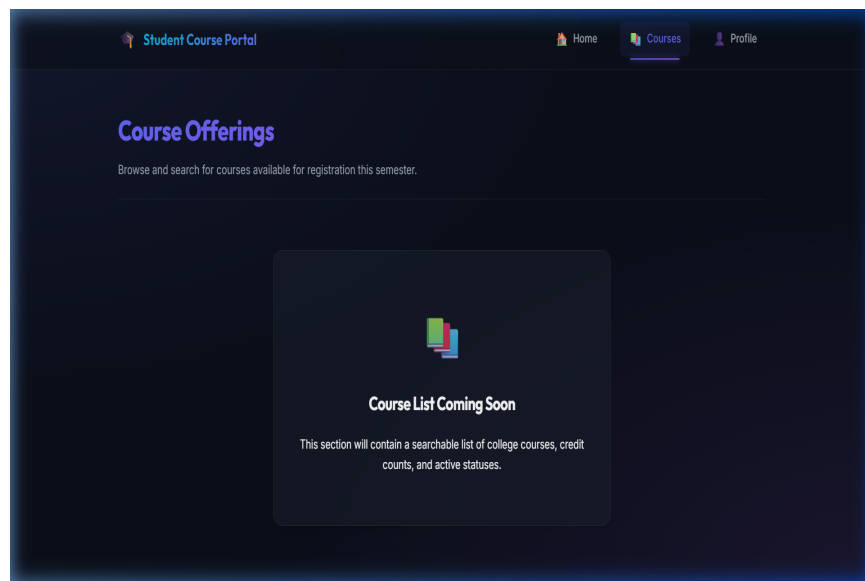
7. Captured Screenshots & Verification

Visual rendering of the portal layout verified on Chrome browser:

Screenshot: Home Dashboard with hardcoded stats (Courses: 12, Enrolled: 3, GPA: 3.8)



Screenshot: Course Offerings list placeholder page loaded via routing



Screenshot: Student Profile placeholder page loaded via routing

Student Profile

Manage your account settings, personal details, and active enrollments.



Student Profile Coming Soon

This section will allow you to view academic transcripts, credit summaries, and cumulative GPA records.

8. Output Explanation

- **Home Page:** Features a dark cyber-slate aesthetic. The header nav remains fixed at the top with a glassmorphism blur. The welcome panel highlights cumulative statistics inside styled gradient cards. Sub-pages are reachable directly via links.
- **Routing:** When clicking 'Courses' or 'Profile', the router intercepts the navigation, updates the active link styles with an animated glowing underscore, and renders the placeholder components instantly without reloading the browser window.
- **Production Build:** Running `npm run build` outputs a production bundle inside `dist/student-course-portal/browser/` containing `index.html`, a single compressed JS bundle (`main-*.js`), and a compiled style sheet (`styles-*.css`). The budget limit checks were triggered and verified successfully.

9. Challenges Faced & Troubleshooting

PowerShell Security Policies: Windows execution policies blocked running ng scripts. *Fix:* Invoked Angular CLI tasks by wrapping with `npx` or overriding execution policies locally.

Angular Standalone Module Structure: The default project scaffolding uses Standalone components (no `app.module.ts`). *Fix:* Resolved compiler issues by manually importing routing directives (`RouterLink`, `RouterLinkActive`) in the Standalone Component imports rather than rely on global declarations.

CSS Layout Responsiveness: Maintaining a clean layout for the stats section on mobile viewports. *Fix:* Used a CSS Grid layout with auto-fit parameters (`repeat(auto-fit, minmax(280px, 1fr))`) to ensure clean responsive wrapping.

10. Learning Outcomes

- ✓ Understood the initialization and directory setup of Angular workspaces.
- ✓ Discovered how modern Angular standalone components are declared and imported.
- ✓ Successfully set up client-side routing using dynamic routing states.
- ✓ Created custom typography and glassmorphic aesthetic interfaces using CSS variables.
- ✓ Inspected bundle structures generated during production compilation.

11. Conclusion

Hands-On 1 successfully establishes a clean, modern, and highly modular starting point for the Student Course Portal. Leveraging the standalone design API and structured routing patterns, the scaffold is ready to support the feature implementations planned for future exercises.